Reinforcement Learning for Autonomous Vehicles: A MetaDrive Implementation Study

James Gunnlaugsson *CPSC 4420 - Artificial Intelligence*
Final Term Project
Clemson, SC
jgunnla@clemson.edu

***Abstract*—This paper works to present the implementation and analysis of an agent to control an autonomous vehicle using reinforcement learning (SAC) and trajectory prediction in the MetaDrive simulator. Through this project, I explored the challenges faced in constructing autonomous driving agents by first implementing and comparing several different RL approaches, leading me to adopt the Soft Actor-Critic (SAC) algorithm after initially experimenting with Proximal Policy Optimization (PPO) and constrained SAC variants. My methodology comprised of three main components: training the agent under SAC to complete basic driving scenarios in a simulated environment, collecting trajectory data from the trained agent, and then developing an LSTM-based prediction model for future trajectory forecasting. Through this, the SAC agent learned how to successfully navigate a three-lane road across 100 various traffic scenarios to almost near perfection. From here, I used the trained agent to collect trajectory data that included vehicle states, actions, and positions, all of which formed the basis for training my prediction model. Lastly, the LSTM predictor worked to achieve accurate trajectories from 10-step historical sequences to predict future 30-step trajectories. The results demonstrate how the combined effort of reinforcement learning (RL) and trajectory prediction can advance autonomous vehicle control while highlighting important considerations for real-world applications and future research.**

I. Introduction

Autonomous driving presents one of today's most intriguing challenges in AI, requiring machines to master the complex decision-making process that human drivers perform instinctively when on the road. Simulation environments like MetaDrive work to offer a testing ground for autonomous driving research, providing users with a controlled space to experiment across different agent implementations and driving scenarios without the risks associated with real-world testing.

This project explores the combination of two key technologies in the field of autonomous driving: reinforcement learning (RL) for an agent's vehicle control and deep learning for trajectory prediction. RL trains an agent to learn optimal behaviors when driving through trial and error, while trajectory predictions adds a predictive element that mirrors a human driver's ability to predict future states.

My work investigates these technologies by testing and training with basic RL approaches and building up a system that can control a vehicle and predict its future trajectory.

II. Related Works

Research on autonomous driving systems (ADS) has highlighted some key challenges in developing reliable control policies. The MetaDrive platform emerged to address the lack of diverse testing environments. While previous simulators like CARLA focused on visual fidelity, MetaDrive prioritizes testing reinforcement learning (RL) algorithms across varied driving conditions [Li et al., 2022].

As time has gone on, RL approaches for autonomous driving have evolved from basic policy gradient methods to more sophisticated algorithms. While PPO (on-policy RL algorithm) gained popularity for its stability in continuous control tasks, SAC (off-policy RL algorithm) had superior sample efficiencies through entropy maximization [Li et al., 2022]. In the case of MetaDrive, SAC has shown promise in its ability to learn from off-policy data while maintaining the desire for exploration. Recent work on constrained RL like SAC-Lagrangian work to incorporate safety constraints during the learning process, though real-world implementations face the challenge of balancing exploration with constraint satisfaction [Li, Peng, Zhao., 2022].

Trajectory predictions are another critical component in ADS. Through the use of a long short-term memory (LSTM) network, researchers Altche and Fortelle demonstrated a model for autonomous vehicles driving on the highway that could predict with accuracy the longitudinal and lateral trajectories (using real driving data from the NGSIM dataset)[Altche el al., 2018]. Their work is important in understanding how LSTM architectures capture temporal patterns in vehicle movement, providing a foundation for predictive models in ADS.

The research presented in this paper bridges from these related works to conduct efficient RL training through SAC with accompanying LSTM-based trajectory predictions, all while making use of MetaDrive's diverse environments and driving scenarios. This approach addresses the limitations that 'pure' RL methods have in anticipating future states while maintaining sample-efficient learning.

III. Method

A. Environment Setup

The implementation of my autonomous driving agent uses MetaDrive to simulate real-world driving environments. My specific environment was configured for single-agent training on a three-lane road, generating 100 different traffic scenarios to train from while maintaining a relatively lower traffic density. For the reward structure of my agent, I modified the function provided in MetaDrive's base environment files and

ran several tweaks throughout my training to produce optimal results. The observation space includes:

- Main Lidar: 180 lasers with 50m range for obstacle detection
- Lane Line Detector: 72 lasers for lane marking detection
- Side Detector: 160 lasers for lateral obstacle detection
- Vehicle State: position, velocity, steering, heading, etc

My environment makes use of the provided node network navigation module, which guides the agent through training by providing relative positions toward future checkpoints along the planned route. This differs from other navigation modules (edge or trajectory-based navigation) by representing the road network as collected nodes, letting the agent be more plan its path in smaller, more manageable segments.

B. SAC Training Implementation

After some initial experiments with PPO showed limitations, I adopted SAC for its off-policy learning capabilities. The SAC agent architecture consists of...

- Twin Q-networks for value estimation
- Policy network for outputting continuous actions (throttle/brake, steering)
- Automatic entropy tuning for exploration
- Experience replay buffer storing state transitions
- Gymnasium-compatible wrapper for integration with stable-baselines3

Because I am importing SAC from stable-baselines3, my implementation extends MetaDriveEnv (provided in base repository) to create a Gymnasium-compatible environment to support SAC training this way.

```
class GymCompatibleMetaDriveEnv(MetaDriveEnv):
   def reset(self, seed=None, options=None):
      if seed is not None:
         self.seed(seed)
      obs, info = super().reset()
      return obs, info
```

C. Trajectory Data Collection

Trajectory collection focuses on using the pre-trained SAC model agent to generate and store driving data (sate information, environment interactions). The collection process runs 100 episodes, each using a unique traffic scenario from our predefined set. The trajectories are saved every 10 episodes in pickle (pkl) format to preserve temporal relationships needed for prediction training. Each scenario is unique in that it generates traffic randomly using the same road layout.

D. LSTM Trajectory Prediction

The prediction system I implemented combines LSTM-based forecasting with GIF visualization done by manipulation with Python's matplotlib library. The predictor first processes stored trajectories into input-output sequences:

```
self.sequences = []
      for traj in self.trajectories:
         positions = traj['positions']
         for i in range(len(positions) -
             (seq_length + pred_length)):
            self.sequences.append({
               'input':
                  positions[i:i+seq_length],
               'target':
                  positions[i+seq_length:i
                  +seq_length+pred_length]
            })
```

My LSTM model works by using a 2-layer architecture with 64 hidden units to recognize temporal patterns. Training samples consist of 10 timesteps of position data to predict the next 30 timesteps. The model then outputs continuous x,y coordinate predictions, which are then visualized via GIF generation.

```
def create_trajectory_gif(model,
    test_trajectory, output_path):
   input_seq = test_trajectory['positions']
       [:10].unsqueeze(0)
   with torch.no_grad():
      predicted_trajectory =
          model(input_seq).squeeze(0).numpy()

   #GIF comparing pred. vs actual paths
   actual =
       test_trajectory['positions'][10:40]
   anim = animation.FuncAnimation(fig,
       animate, frames=30, interval=100)
   anim.save(output_path, writer='pillow')
```

This visualization system generates 30-frame animations at 100ms intervals while plotting both the predicted and actual trajectories for comparison. A 10-timestep input and 30-timestep prediction were chosen to balance giving the LSTM model too much context for accurate forecasting while also maintaining computational complexity. GIFs are generated every 10th trajectory file to provide visual validation of the model's performance.

IV. Experiments and Results

A. Training Performance

The training utilized SAC over 250,000 total environment steps (with max steps set at 1000 per episode), chosen for its off-policy efficiency over PPO. The SAC implementation used stable-baselines3's framework with MlpPolicy and my custom Gymnasium-compatible environment wrapper.

```
def train_sac(env, timesteps=2500000):
   check_env(env.envs[0])
   model = SAC("MlpPolicy", env, verbose=1,
       tensorboard_log="./sac_metadrive/")
   callback = ProgressCallback(timesteps)
   model.learn(total_timesteps=timesteps,
       callback=callback)
   model.save("sac_metadrive_agent")
   print("Model training complete and saved
       as sac_metadrive_agent.")
   return model
```

B. Model Evaluation

Evaluation across 100 test episodes demonstrated robust performance for my agent:

- Success rate: 85%
- Avg. Safety Violations: 0.17 ± 0.38
- Avg. Return: 215.61 ± 28.99

The agent showed consistent ability to navigate straight paths and avoid obstacles, with minimal safety violations indicating good adherence to the well-rewarded driving rules.

C. Trajectory Prediction Analysis

The LSTM prediction system processed 10-timestep sequences to generate 30-timestep trajectory forecasts. While the predictions maintained reasonable accuracy during normal driving, analysis revealed limitations in the implementation:

The visualizations, generated through GIFs, showed nearly identical predictions between episodes 10 and 100, suggesting some underfitting or limited variation in the training data. This could stem from consistent driving behavior from the SAC agent, insufficient diversity in collected trajectories, or potential limitations in the LSTM architecture for capturing subtle variations. The prediction system demonstrated the basic ability to predict straight-line trajectories, speed variations, steering maneuvers, etc.

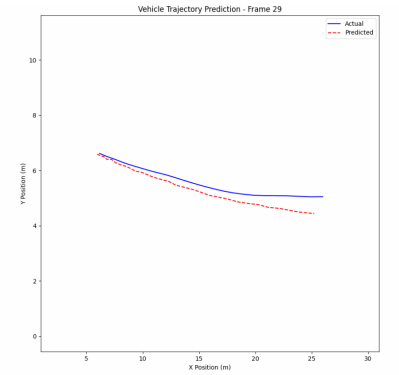

Fig. 1. Trajectory prediction after 10 episodes.

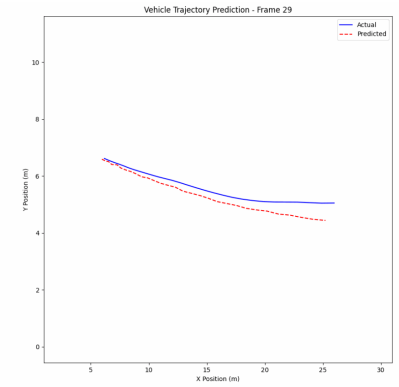

Fig. 2. Trajectory prediction after 100 episodes.

However, the lack of variation between predictions suggests room for improvement through increased diversity in training data or enhancing the current LSTM architecture.

V. Conclusion

This project trained and evaluated an agent in an autonomous driving setting, combining RL with trajectory prediction. Using the MetaDrive simulator, I successfully trained an SAC agent that achieved an 85% success rate across 100 unique scenarios, with minimal safety violations (0.17 ± 0.38).

The LSTM-based trajectory prediction model, though functional, revealed opportunities for future improvement. The similarity across different episodes suggests that I could improve my predictions through a variety of factors, such as implementing better data collection strategies, increasing trajectory data diversity, enhancing the overall LSTM architecture, etc.

As for lessons learned, this work taught me the importance of proper environment configuration for stable training, the benefits of off-policy learning for autonomous driving systems, the challenges in balancing exploration with safety constraints, and the need for diverse training scenarios. Future improvements should include implementing a SAC-Lag agent for better safety guarantees, expanding to multi-agent scenarios, and developing more sophisticated trajectory prediction models that better capture driving behavior variations.

References

[1] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue and B. Zhou, "MetaDrive: Composing Diverse Driving Scenarios for Generalizable Reinforcement Learning," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 45, no. 3, pp. 3461-3475, 1 March 2023

[2] I. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in Magnetism, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.

[3] Li, Q., Peng, Z., Zhou, B. (2022). Efficient Learning of Safe Driving Policy via Human-AI Copilot Optimization. ArXiv. https://arxiv.org/abs/2202.10341